

Route AI/ML Final Projects

1. Introduction to Projects

This document outlines a set of machine learning, computer vision, and NLP projects. Each project follows a standard development pipeline and must be delivered as a working application with a Streamlit interface.

1.1 How the projects are split

Projects are grouped into three difficulty levels: Easy, Medium, and Hard. The difficulty affects the expected complexity of the model, data preprocessing, and deployment challenges. Each project also includes an estimated team size ranging from 1 to 3 members, based on the difficulty level.

- **Easy projects** are suitable for 1-2 members.
- **Medium projects** need 2-3 members.
- **Hard projects** need 3 members.

1.2 Project Structure

Each team must deliver two main components:

1. **Jupyter Notebook** that contains all experiments, data exploration, preprocessing, model training, evaluation, and model export.
2. **Github Repo Link** with Python codebase with a Streamlit application that loads the exported model and provides an interactive interface for predictions.

All code must be version controlled on GitHub. Deployment on Streamlit Cloud is optional but recommended as a bonus.

2. Shared Requirements and Final Deliverables

2.1 Jupyter Notebook Guide (Experiments and Pipeline)

The notebook should include these sections in order:

- **Importing modules:** All required libraries.
- **Loading data:** From a public source or a local file.
- **Analysis and Visualization:** Exploratory data analysis, class distribution, sample inspection, and relevant plots based on dataset domain (Tabular data, Text, Images, etc.).

- **Preprocessing:** Handling missing values, scaling, encoding, text cleaning, image resizing, data augmentation, etc.
- **Model building:** Baseline model, hyperparameter tuning, cross-validation (if applicable), final model training.
- **Evaluation:** Metrics appropriate to the task (accuracy, F1, IoU, BLEU, etc.) with justification.
- **Exporting the model:** Save the final model in a standard format (`.pkl` , `.h5` , `.pt` , `.keras` , `.onnx` , or a folder with model + preprocessor).

2.2 Streamlit Application Codebase (Suggested Structure)

```
project_root/
├─ app.py
├─ model/
│   ├─ __init__.py
│   ├─ predict.py
│   └─ preprocess.py
├─ requirements.txt
├─ README.md
└─ .gitignore
```

- `app.py` : Streamlit entry point. Contains UI widgets, file uploaders, camera inputs, and calls the prediction function.
- `model/predict.py` : Loads the exported model and implements an `inference(data)` function that returns predictions.
- `model/preprocess.py` : Contains the needed preprocessing logic.
- `requirements.txt` : Lists all dependencies (e.g., `streamlit` , `scikit-learn` , `tensorflow` , etc.).

The Streamlit interface must allow user input (text, numbers, file upload, image capture, etc.) and display predictions with confidence or explanation where feasible.

2.3 GitHub Submission

- The repository must be public.
- The `README.md` must include:
 - Project description
 - Link to demo / deployed app (if any)
 - How to run locally (commands)
 - Used Dataset

2.4 Deployment on Streamlit Cloud (Bonus)

- Connect the GitHub repository to Streamlit Cloud.

- Set `app.py` as the entry point.
- The app must be accessible via a public URL.

2.5 Team Contribution

A `CONTRIBUTIONS.md` file or a section in `README.md` must detail what each member did.

3. Projects

Below is the complete list of projects. Each entry includes difficulty level, team size, a brief description, and hints about expected approach, tools, and challenges.

3.1 Salary Prediction

- **Difficulty** : Easy
- **Team size** : 1-2
- **Description** : Predict an employee's salary based on features such as years of experience, education level, job title, and location.

3.2 Car Price Prediction

- **Difficulty** : Easy to Medium
- **Team size** : 1-2
- **Description** : Estimate the market price of a used car given attributes like brand, model year, mileage, fuel type, and transmission.
- **Hints** :
 - Regression model with feature engineering (e.g., car age = current year - model year).

3.3 Customer Churn Prediction

- **Difficulty** : Medium
- **Team size** : 2-3
- **Description** : Predict whether a customer will stop using a service (e.g., telecom, banking) based on usage patterns, account information, and support interactions.
- **Hints** :
 - Binary classification with imbalanced data. Use SMOTE or class weighting.
 - Dataset: Can be Telco Customer Churn (IBM) or Bank Churn dataset.

3.4 Customer Segmentation

- **Difficulty** : Medium
- **Team size** : 2

- **Description** : Group customers into distinct segments based on purchasing behavior, demographics, or website activity.
- **Hints** :
 - Unsupervised learning using K-Means, DBSCAN, or Gaussian Mixture Models.
 - Dataset: Online Retail (UCI) or Mall Customer Segmentation.

3.5 Business Card Scanner (OCR)

- **Difficulty** : Medium
- **Team size** : 2-3
- **Description** : Extract text (name, phone, email, company) from an uploaded image of a business card.
- **Hints** :
 - Use Tesseract OCR (pytesseract) with OpenCV for preprocessing (grayscale, thresholding, deskewing).
 - Optional: Use regex to parse fields (Extracting pattern based information like emails, phone numbers and more independently).
 - Dataset: Public business card datasets (e.g., from Kaggle or Roboflow).
 - Challenges: Varying layouts, fonts, and image quality; text localization.
 - Tools: OpenCV, pytesseract, Streamlit file uploader.

3.6 Virtual On-Screen Keyboard Typing

- **Difficulty** : Hard
- **Team size** : 3
- **Description** : Create a virtual keyboard displayed on screen. Users type by pointing a finger or using hand gestures over the webcam.
- **Hints** :
 - Hand landmark detection (MediaPipe Hands or OpenCV + custom model). Map finger position to keys.
 - Real-time performance required.
 - Dataset: None; use MediaPipe pretrained model.
 - Challenges: Latency, calibration, robust key activation (hover vs click), handling varying hand sizes.
 - Tools: OpenCV, MediaPipe, Streamlit with video streaming (cv2).

3.7 Hand Gestures Recognition

- **Difficulty** : Medium to Hard
- **Team size** : 2-3 (3 for dynamic gestures)
- **Description** : Recognize static hand gestures (e.g., peace, thumbs up) or dynamic gestures (swipe, pinch) from live webcam feed.

- **Hints :**
 - For static: Use MediaPipe landmarks + a classifier (ML or DL Based).
 - For dynamic: Use LSTM or 3D CNN on landmark sequences.
 - Dataset: HaGRID, 20bn-jester.
 - Challenges: Lighting invariance, background clutter, real-time processing.
 - Tools: MediaPipe, TensorFlow/PyTorch, OpenCV, Streamlit.

3.8 Emotion Detection (Face)

- **Difficulty :** Medium to Hard
- **Team size :** 2-3
- **Description :** Detect human faces in an image or webcam feed and classify the emotion (happy, sad, angry, surprised, neutral, etc.).
- **Hints :**
 - Use a pretrained model: FER (Facial Expression Recognition) or DeepFace.
 - Face detection: Haar cascades, MTCNN, or MediaPipe Face Detection.
 - Dataset: FER2013, AffectNet.
 - Challenges: Real-time performance, varying lighting and head poses.
 - Tools: OpenCV, DeepFace library, TensorFlow, Streamlit.

3.9 Traffic Light Detection

- **Difficulty :** Hard
- **Team size :** 3
- **Description :** Detect traffic lights and classify their state (red, yellow, green) from dashcam images or live video.
- **Hints :**
 - Object detection + classifier. Use pretrained model (YOLOv8) fine-tuned on traffic light datasets (e.g., Bosch Small Traffic Lights, LISA).
 - Challenges: Small objects, varying weather and lighting.
 - Tools: YOLOv8, OpenCV, Streamlit with video upload or webcam.

3.10 Article Summarization Tool

- **Difficulty :** Medium
- **Team size :** 2
- **Description :** Generate a short summary of a news article or blog post.
- **Hints :**
 - Extractive summarization (TextRank, BERT Extractive) is easier; abstractive (BART, T5, Seq2Seq Custom Model) gives better results but requires more resources (Recommended).
 - Dataset: CNN/Daily Mail, XSum, or use a pretrained model.
 - Challenges: Coherence, handling long documents.

- Tools: Hugging Face transformers (pipeline "summarization"), Streamlit text area.

3.11 News Categorization Model

- **Difficulty** : Easy to Medium
- **Team size** : 1-2
- **Description** : Categorize news articles into predefined topics such as sports, politics, technology, health, etc.
- **Hints** :
 - Fine-tune DistilBERT (Recommended) or use a traditional TF-IDF + Linear SVM.
 - Dataset: BBC News, AG News, or Reuters.
 - Challenges: Multi-class classification, overlapping categories.
 - Tools: scikit-learn, Hugging Face, Streamlit for text input or URL parsing.

3.12 Spam Email Detection

- **Difficulty** : Easy
 - **Team size** : 1-2
 - **Description** : Classify emails as spam or ham (non-spam) based on email text and metadata.
 - **Hints** :
 - Classic NLP with Naive Bayes (MultinomialNB) and TF-IDF works very well.
 - Dataset: Enron Spam, SpamAssassin, or UCI SMS Spam Collection.
 - Challenges: Avoiding overfitting on specific keywords.
 - Tools: scikit-learn (CountVectorizer, TfidfVectorizer), Streamlit text input.
-

4. Final Notes

Teams must select one project from the list above. Make sure to deliver all mandatory shared requirements in Section 2. The bonus deployment on Streamlit Cloud is encouraged but not required for full marks. Good luck!